UNIT - 4
BASICS OF R

Prashant Kothari
Assistant Professor
Computer Science and Engineering

Contents

- R Basics
- Data types and objects
- Control structures
- Data frame
- Feature Engineering - scaling, Label
- Encoding and One Hot Encoding
- Reduction

R Basics

R is a popular programming language used for **statistical computing** and **graphical presentation**.

Its most common use is to analyze and visualize data. Today, it is one of the most popular programming languages in data science and research.

R was developed by **Ross Ihaka** and **Robert Gentleman** in **1992**.

The **first version of R** was released publicly in **1993**. It was made available under the **GNU General Public License (GPL)**, allowing anyone to use, modify, and distribute it freely. R became an **official open-source** project in **1995**.

Software Required

1. RStudio
2. R



Why Was R Developed?

The objectives of developing R were:

- To provide a **free and open-source** statistical programming language.
- To perform **statistical analysis** efficiently.
- To create **high-quality data visualizations**.
- To support **research and academic work**.
- To make statistical tools accessible to everyone.

Why use R ?

- ✓ It is a great resource for **data analysis, data visualization, data science** and **machine learning**.
- ✓ It provides many **statistical techniques** (such as statistical tests, classification, clustering and data reduction).
- ✓ It is easy to **draw graphs** in R, like pie charts, histograms, box plot, scatter plot, etc.
- ✓ It works on **different platforms** (Windows, Mac, Linux).
- ✓ It is **open-source, free** and it has a **large community** support.
- ✓ It has **many packages** (libraries of functions) that can be used to solve different problems.

Applications of R

1. Data Analysis
2. Statistical Modeling
3. Machine Learning
4. Artificial Intelligence
5. Data Visualization
6. Predictive Analytics
7. Financial Analysis
8. Bioinformatics
9. Healthcare Analytics
10. Academic Research

Comment in R

Comments can be used to explain R code and to make it more readable.

It can also be used to prevent execution when testing alternative code.

Comments starts with a #. When executing code, R will ignore anything that starts with **#**.

Example:

```
# This is a comment
"Hello World!“
```

OR

```
"Hello World!"    # This is a comment
```

Data Types in R

Data types define the type of data stored in variables.

Data Type	Description	Example
Numeric	Decimal values	contains any **number with or without a decimal**, like - 10.5, 55, 78710.5, 787
Integer	Whole numbers	Integers are numeric data without decimals. where the **letter "L" declares this as an integer**. Like- 1**L**, 55**L**, 100**L**
Character	Text/String	“k", "R is exciting", "FALSE", "11.5"
Logical	TRUE/FALSE	TRUE or FALSE (Boolean)
Complex	Complex numbers	9 + 3**i**, where "**i**" is the imaginary part

Data Types in R

Data type of variable – use **class(variable name)**

Example:

```
x <- 4.2
Y <- 509L
# Print the class name of x and y
class(x)      # output - numeric
class(y)      # output - integer
```

Data Types in R

Example:

```
x <- 10.5          # numeric
print(x)

y <- 1000L         # integer
print(y)

z <- 9 i + 3       # complex
print(z)
```

Data Types in R

Example:

```
m <- "R is exciting“          # character/string
print(m)

k <- TRUE                     # logical/boolean
print(k)
```

Types conversion

You can convert from one type to another with the following functions.

1. as.numeric()
2. as.integer()
3. as.complex()

Types conversion

Example:

```
x <- 1L          # integer
y <- 2           # numeric

# convert from integer to numeric:
a <- as.numeric(x)

# convert from numeric to integer:
b <- as.integer(y)

# After conversion - print the class name of a and b
class(a)         # numeric
class(b)         # integer
```

Variable

Variables are containers for storing data values.

R does not have a command for declaring a variable. A variable is created the moment you first assign a value to it. To assign a value to a variable, use the **<- sign**. To output (or **print()**) the variable value, just type the variable name.

Example:

```
name <- "John"
age <- 40

name    OR   print(name)        # output "John"
age     OR   print(age)         # output 40
```

Variable

Concatenate Elements

concatenate or join two or more elements, by using the **paste()** function.

To combine both text and a variable, R uses comma (**,**).

Example-

```
text <- "awesome"

paste("R is", text)
```

OR

```
text1 <- "R is"
text2 <- "awesome"

paste(text1, text2)
```

Variable

Concatenate Elements

Example-

```
num1 <- 5

num2 <- 10

num1 + num2
```

Variable

Multiple Variable

R allows you to **assign the same value to multiple variables** in one line.

Example-

```
var1 <- var2 <- var3 <- "Orange"
```

Operators

Operators are used to perform operations on variables and values.

R divides the operators in the following groups:

1. Arithmetic operators
2. Assignment operators
3. Comparison operators
4. Logical operators
5. Miscellaneous operators

Operators

1. Arithmetic operators

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
^	Exponent	x ^ y
%%	Modulus (Remainder from division)	x %% y
%/%	Integer Division	x%/%y

Operators

2. Assignment operators

Assignment operators are used to assign values to variables.

Example

my_var <- 3 **OR** 3 -> my_var

my_var <<- 3 **OR** 3 ->> my_var

my_var # print my_var

Operators

2. Assignment operators

<<- is a **global assigner**.

It is also possible to turn the direction of the assignment operator.

Example:

x <- 3 is equal to **3 -> x**

Operators

3. Comparison operators

Comparison operators are used to compare two values.

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Operators

4. Logical operators

Logical operators are used to combine conditional statements.

Operator	Description
&&	Logical AND operator - Returns TRUE if both statements are TRUE
||	Logical OR operator. Returns TRUE if one of the statements is TRUE
!	Logical NOT - Returns FALSE if statement is TRUE

Operators

5. Miscellaneous operators

Miscellaneous operators are used to manipulate data.

Operator	Description	Example
:	Creates a series of numbers in a sequence	x <- 1:10
%in%	Find out if an element belongs to a vector	x %in% y
%*%	Matrix Multiplication	x <- Matrix1 %*% Matrix2

Operators

5. **Miscellaneous operators**

: (Colon Operator) –

Creates a sequence of consecutive numbers.

Example

```
x <- 1:10
print(x)          #Output -  [1] 1 2 3 4 5 6 7 8 9 10
y <- 10:1
print(y)          #Output -  [1] 10 9 8 7 6 5 4 3 2 1
```

Operators

5. Miscellaneous operators

%in% (Membership Operator) –

Checks whether an element exists in a vector and returns TRUE or FALSE.

Example

```
x <- c(10,20,30,40,50)
20 %in% x                    #Output - [1] TRUE

x <- c(10,20,30,40,50)
100 %in% x                   #Output - [1] FALSE

students <- c(101,102,103,104,105)
103 %in% students                    #Output - [1] TRUE
```

Operators

5. Miscellaneous operators

%*% (Matrix Multiplication Operator) - Performs matrix multiplication

Example

```
A <- matrix(c(1,2,3,4), nrow=2)
B <- matrix(c(5,6,7,8), nrow=2)
     [,1] [,2]          #Matrix A
[1,]    1    3
[2,]    2    4
     [,1] [,2] ]        #Matrix B
[1,]    5    7
[2,]    6    8
```

Operators

5. **Miscellaneous operators**

%*% (Matrix Multiplication Operator)

Example

```
C <- A %*% B

print(C)
```

#Output

```
     [,1] [,2]      #Matrix C
[1,]   23   31
[2,]   24   46
```

Control structures

If Statements

An "if statement" is written with the **if** keyword and it is used to specify a block of code to be executed if a condition is **TRUE**.

Example:

```
a <- 33
b <- 200

if (b > a)
{
    print("b is greater than a")
}
```

control structures

If Statements

An "if statement" is written with the if keyword, and it is used to specify a block of code to be executed if a condition is **TRUE**.

Example:

```
a <- 33
b <- 200

if (b > a)
{
    print("b is greater than a")
}
```

NOTE: R uses curly brackets { } to define the scope in the code.

control structures

If else Statements

You can also use else without else if.

Example:

```
a <- 200
b <- 33

if (b > a) {
        print("b is greater than a")
 } else {
        print("b is not greater than a")
}
```

control structures

Nested If else Statements

The **else** keyword catches anything which isn't caught by the preceding conditions

Example:

```
a <- 331
b <- 200
if (b > a) {
      print("b is greater than a")
} else if (a == b) {
            print("a and b are equal")
         } else {
                     print("a is greater than b")
 }
```

Loop control structures

Loops can execute a block of code as long as a specified condition is reached.

Loops are handy because they save time, reduce errors and they make code more readable.

R has two loop commands:

1. while loops
2. for loops

Loop control structures

1. While

With the while loop we can execute a set of statements as long as a condition is TRUE:

Example:

```
i <- 1
while (i < 6) {
print(i)
i <- i + 1
}
```

Loop control structures

1. While

While with break.

Example:

```
i <- 0
while (i < 6) {
i <- i + 1
if (i == 3) {
  next
}
print(i)
}
```

Loop control structures

2. for

A for loop is used for iterating over a sequence:

Example:

```
for (x in 1:10) {
print(x)
}
```

Function

A function is a block of code which only runs when it is called. You can pass data, known as parameters into a function.

A function can return data as a result. To create a function, use the **function()** keyword.

Create a Function

Example

```
my_function <- function() {
print("Hello World!")
}
```

Call a Function

To call a function, use the function name followed by parenthesis, like my_function():

Example

```
my_function()
```

Passing parameter into Function

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses.

You can add as many arguments as you want, just separate them with a comma.

Example

```
my_function <- function(fname, lname) {
paste(fname, lname)
}

my_function("Peter", "Griffin")
```

Function Return Values

To let a function return a result, use the return() function.

Example

```
my_function <- function(x) {
  return (5 * x)
}

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

Global variable in function

Variables that are created outside of a function are known as **global** variables.

Global variables can be used by everyone, both inside of functions and outside.

Example

```
txt <- "awesome"
my_function <- function() {
paste("R is", txt)
}
my_function()
```

Data frame

A Data Frame is one of the most commonly used data structures in R programming. It is used to store data in a **tabular format**, similar to an Excel spreadsheet or a database table.

A data frame consists of:

Rows → Represent observations or records.

Columns → Represent variables or attributes.

Different columns can contain **different data types** such as numeric, character, logical etc.

1. Creating a Data Frame

The **data.frame()** function is used to create a data frame

Example

Suppose we have student information.

```
students <- data.frame(
RollNo = c(101, 102, 103, 104, 105),
Name = c("Rahul", "Amit", "Neha", "Priya", "Raj"),
Marks = c(85, 72, 91, 35, 68),
Result = c("Pass", "Pass", "Pass", "Fail", "Pass")
)
print(students)
```

1. Creating a Data Frame

Output

	RollNo	Name	Marks	Result
1	101	Rahul	85	Pass
2	102	Amit	72	Pass
3	103	Neha	91	Pass
4	104	Priya	35	Fail
5	105	Raj	68	Pass

2. Summarize the Data

Use the **summary()** function to summarize the data from a Data Frame.

Example

```
students <- data.frame(
RollNo = c(101, 102, 103, 104, 105),
Name = c("Rahul", "Amit", "Neha", "Priya", "Raj"),
Marks = c(85, 72, 91, 35, 68),
Result = c("Pass", "Pass", "Pass", "Fail", "Pass")
)
Summary(students)
```

3. Data Frame Length

Use the **length()** function to find the number of columns in a Data Frame (similar to ncol()).

Example

```
students <- data.frame(
RollNo = c(101, 102, 103, 104, 105),
Name = c("Rahul", "Amit", "Neha", "Priya", "Raj"),
Marks = c(85, 72, 91, 35, 68),
Result = c("Pass", "Pass", "Pass", "Fail", "Pass")
)
length(students)
```

4. Combining Data Frames

Use the rbind() function to combine two or more data frames in R vertically.

Example

```
students1 <- data.frame(
RollNo = c(101, 102),
Name = c("Rahul", "Amit"),
Marks = c(85, 72),
Result = c("Fail", "Pass")
)
```

4. Combining Data Frames

Use the rbind() function to combine two or more data frames in R vertically.

Example

```
students2 <- data.frame(
RollNo = c(201, 202),
Name = c(“Kamal", “Hari"),
Marks = c(74,12),
Result = c(“Pass”, "Pass")
)
```

4. Combining Data Frames

Use the **rbind()** function to combine two or more data frames in R vertically.

Example

```
New_Data_Frame <- rbind(students1, students2)
New_Data_Frame
```

5. Add Columns

Use the **cbind()** function to add new columns in a Data Frame.

Example

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)

New_col_DF <- cbind(Data_Frame, Steps = c(1000, 6000, 2000))
New_col_DF
```

6. Add Row

Use the **rbind()** function to add new rows in a Data Frame.

Example

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)


New_row_DF <- rbind(Data_Frame, c("Strength", 120, 90))
New_row_DF
```

7. Access Items

We can use single brackets [ ], double brackets [[ ]] or $ to access columns from a data frame.

Example

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)
Data_Frame[1]
Data_Frame[["Training"]]
Data_Frame$Training
```

8. Remove Rows and Columns

Use the **c()** function to remove rows and columns in a Data Frame.

Example

```
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)
Data_Frame_New <- Data_Frame[-c(1), -c(1)]
Data_Frame_New
```

Feature Engineering

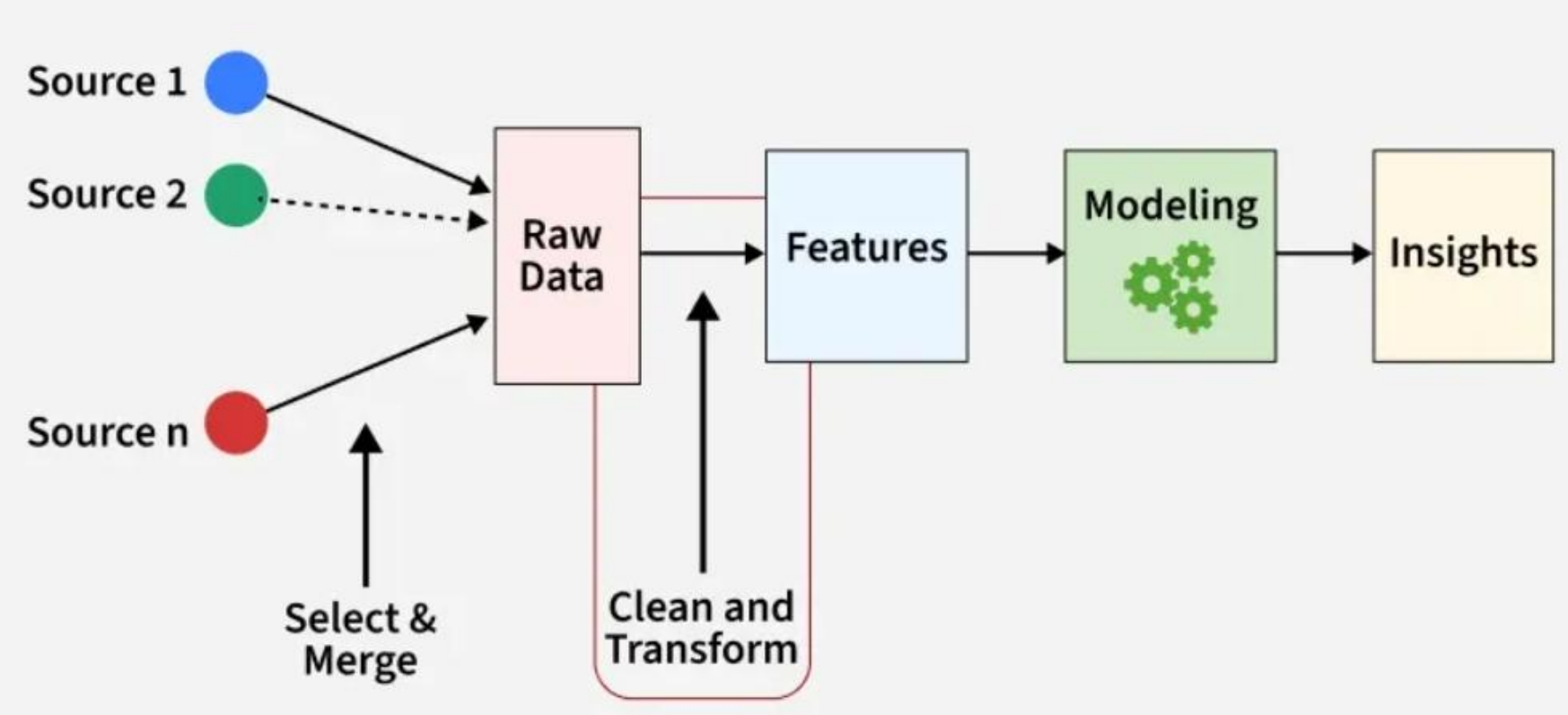
A feature is an input variable used by a Machine Learning model to make predictions.

For example - suppose we want to predict whether a student will pass or fail.

Feature Engineering is the process of creating, transforming, selecting or extracting useful features from raw data so that Machine Learning models can learn more effectively.

In simple words - Feature Engineering means converting raw data into meaningful information that can be better understood by a Machine Learning algorithm.

Feature Engineering



Importance of Feature Engineering

1. **Improve accuracy:** Choosing the right features helps the model learn better, leading to more accurate predictions.
2. **Reduce overfitting:** It is an important features that helps the model avoid memorizing the data.
3. **Boost interpretability:** Well-chosen features make it easier to understand how the model makes its predictions.
4. **Enhance efficiency:** Focusing on key features speeds up the model's training and prediction process, saving time and resources.

Feature Engineering Process



Feature Engineering Process

1. Feature Creation: Feature creation involves generating new features from domain knowledge or by observing patterns in the data.

- **Domain-specific**: Created based on industry knowledge like business rules.
- **Data-driven**: Derived by recognizing patterns in data.
- **Synthetic**: Formed by combining existing features.

Feature Engineering Process

2. Feature Transformation: Transformation adjusts features to improve model learning:

- **Normalization & Scaling**: Adjust the range of features for consistency.
- **Encoding**: Converts categorical data to numerical form i.e one-hot encoding.
- **Mathematical transformations**: Like logarithmic transformations for skewed data.

Feature Engineering Process

3. Feature Extraction: Transform existing features into a lower-dimensional or more informative representation (e.g., PCA).

- **Dimensionality reduction**: Techniques like PCA reduce features while preserving important information.
- **Aggregation & Combination**: Summing or averaging features to simplify the model.

Feature Engineering Process

4. Feature Selection: Feature selection involves choosing a subset of relevant features to use:

- **Filter methods**: Based on statistical measures like correlation.
- **Wrapper methods**: Select based on model performance.
- **Embedded methods**: Feature selection integrated within model training.

Feature Engineering Process

5. Feature Scaling: Scaling ensures that all features contribute equally to the model:

- **Min-Max scaling**: Rescales values to a fixed range like 0 to 1.
- **Standard scaling**: Standardizes features to have mean 0 and variance 1.

Steps in Feature Engineering

Feature engineering can vary depending on the specific problem but the general steps are:

1. **Data Cleaning:** Identify and correct errors or inconsistencies in the dataset to ensure data quality and reliability.
2. **Data Transformation:** Transform raw data into a format suitable for modeling including scaling, normalization and encoding.
3. **Feature Extraction:** Create new features by combining or deriving information from existing ones to provide more meaningful input to the model.

Steps in Feature Engineering

4. **Feature Selection:** Choose the most relevant features for the model using techniques like correlation analysis and stepwise regression.
5. **Feature Iteration:** Continuously refine features based on model performance by adding, removing or modifying features for improvement.

Feature Scaling

Feature Scaling is the process of transforming numerical features so that they are brought to a similar or comparable scale.

Consider the following dataset:

Age	Salary
20	25,000
30	50,000
40	100,000

Scaling converts them into comparable ranges.

Feature Scaling

Here:

- Age ranges from 20–40
- Salary ranges from 25,000–100,000

Because the numerical ranges are very different, some algorithms may give more importance to Salary simply because its numbers are larger. Scaling helps solve this problem.

Why is Scaling Required?

Feature scaling is particularly important for algorithms that use:

- Distance
- Gradient-based optimization
- Variance/covariance

Examples:

- ✓ K-Nearest Neighbors (KNN)
- ✓ K-Means Clustering
- ✓ Support Vector Machine (SVM)
- ✓ Logistic Regression
- ✓ Neural Networks
- ✓ PCA

Why is Scaling Required?

Example

Suppose we have two features:

Age = 25

Salary = 80000

If a distance calculation is performed directly, the salary value can have a much greater numerical influence.

After scaling:

Age → 0.25

Salary → 0.31

Both features now have comparable magnitudes.

Why is Scaling Required?

1.1 Standardization

Standardization is also called **Z-score normalization**.

It transforms the data using:

$$z = \frac{x - \mu}{\sigma}$$

Where:

- x\x = original value
- μ\mu = mean
- σ\sigma = standard deviation
- z\z = scaled value

Why is Scaling Required?

1.2 Min-Max Scaling

Min-Max Scaling converts values to a specified range, commonly **0 to 1**.

Formula:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Label Encoding

Label Encoding converts categorical values into numerical values.

Machine Learning algorithms often require numerical input. Therefore, text categories need to be represented numerically.

Example

Suppose:

Gender
Male
Female
Male
Female

After Label Encoding:

Gender	Encoded Value
Male	1
Female	0
Male	1
Female	0

When Should Label Encoding Be Used?

Label Encoding is particularly appropriate when categories have a **natural order**.

For example: Low, Medium, High

We can represent them as:

Low → 0

Medium → 1

High → 2

Here, the numerical order represents a meaningful relationship:

Low < Medium < High

One-Hot Encoding

One-Hot Encoding converts categorical values into **separate binary columns.**

Suppose:

City
Delhi
Mumbai
Vadodara

After One-Hot Encoding:

Delhi	Mumbai	Vadodara
1	0	0
0	1	0
0	0	1

Each category gets its own column.

Label Encoding vs One-Hot Encoding

Label Encoding	One-Hot Encoding
Converts categories into numbers	Converts categories into binary columns
Usually produces one encoded column	Produces multiple columns
Can represent ordinal categories	Best suited to nominal categories
May imply an order	Does not imply an order
Example: Low → 0, Medium → 1, High → 2	Example: Delhi → [1,0,0]

Reduction

Dimensionality Reduction is the process of reducing the number of features or dimensions in a dataset while retaining as much useful information as possible.

For example:

100 Features

↓

Dimensionality Reduction

↓

20 Features

The goal is to make the dataset simpler without losing too much important information.

Why Do We Need Dimensionality Reduction?

Large datasets can contain hundreds or thousands of features.

If we use all these features:

- ✓ Computation becomes expensive.
- ✓ Training can take longer.
- ✓ Some features may be redundant.
- ✓ Noise may increase.
- ✓ Visualization becomes difficult.
- ✓ Models may become more prone to overfitting.
- ✓ The **curse of dimensionality** can affect some algorithms.

Two Main Approaches to Dimensionality Reduction

1. Feature Selection
2. Feature Extraction

Two Main Approaches to Dimensionality Reduction

1. Feature Selection

Feature Selection means selecting only the most important features from the original dataset and removing irrelevant, redundant or less useful features.

Suppose we have a student-performance dataset:

Two Main Approaches to Dimensionality Reduction

1. Feature Selection

Feature	Description
Study Hours	Hours studied
Attendance	Attendance percentage
Previous Marks	Previous exam marks
Student ID	Unique ID
Favorite Color	Favorite color
Final Result	Target variable

Two Main Approaches to Dimensionality Reduction

1. Feature Selection

For predicting the **final result**, features such as **Study Hours, Attendance** and **Previous Marks** may be useful, while **Student ID and Favorite Color** may not contribute much.

After feature selection:

The important point is that Feature Selection **does not create new features**. It simply chooses useful features from the existing ones.

Two Main Approaches to Dimensionality Reduction

1. **Feature Selection**

Types of Feature Selection

There are three common methods of Feature Selection:

A. Filter Methods

B. Wrapper Methods

C. Embedded Methods

Two Main Approaches to Dimensionality Reduction

2. Feature Extraction

Feature Extraction is different from Feature Selection.

Instead of selecting some existing features, Feature Extraction **creates new features by transforming** or **combining the original features**.

The most commonly used Feature Extraction technique is **Principal Component Analysis (PCA)**.

PCA converts highly correlated original variables into a smaller number of new variables called **Principal.**

Difference between Feature Selection vs Feature Extraction

Feature Selection	Feature Extraction
Selects existing features	Creates new features
Original features remain unchanged	Original features are transformed
Easy to interpret	Usually harder to interpret
Removes unnecessary features	Combines information from features
Examples: RFE, Chi-Square, Lasso	Examples: PCA, LDA, Autoencoders
Feature names remain meaningful	New features may not have direct meaning

Difference between Feature Selection vs Feature Extraction

In short- remember this distinction:

Feature Selection = Choose the best existing features

Feature Extraction = Create fewer new features from the existing features

Both approaches aim to reduce dimensionality, improve efficiency, reduce model complexity and help the model generalize better.

Reference Books

1. **Data Science for Business** – Foster Provost and Tom Fawcett, *Data Science for Business*, O'Reilly Media, 2013.
2. **Python for Data Analysis** – Wes McKinney, *Python for Data Analysis*, 2nd Edition, O'Reilly Media, 2017.
3. **Doing Data Science** – Cathy O'Neil and Rachel Schutt, *Doing Data Science*, O'Reilly Media, 2013.
4. **Data Science from Scratch** – Joel Grus, *Data Science from Scratch*, 2nd Edition, O'Reilly Media, 2019.
5. **Big Data Fundamentals** – Thomas Erl, Wajid Khattak, and Paul Buhler, *Big Data Fundamentals*, Prentice Hall, 2015.

Parul® University | NAAC GRADE A++

https://paruluniversity.ac.in/

